

LAB | Intro to Machine Learning

Solved version — Spaceship Titanic

Goal: build a first supervised Machine Learning model that predicts whether a passenger was **Transported** to another dimension, using the workflow seen in class:

split → instantiate → fit → predict → score

Load the data

In this challenge, we will be working with Spaceship Titanic data. The data can be found here:

https://raw.githubusercontent.com/data-bootcamp-v4/data/main/spaceship_titanic.csv

Metadata

https://github.com/data-bootcamp-v4/data/blob/main/spaceship_titanic.md

```
In [1]: # import libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
In [2]: spaceship = pd.read_csv("https://raw.githubusercontent.com/data-bootcamp-v4/
spaceship.head()
```

```
Out[2]:
```

	PassengerId	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService
0	0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0
1	0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0
2	0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0
3	0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0
4	0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0

Check the shape of your data

`.shape` returns `(rows, columns)`. Always the first thing to check: how much data do we have to learn from?

```
In [3]: spaceship.shape
```

```
Out[3]: (8693, 14)
```

Check for data types

This matters a lot for KNN: the model only accepts **numbers**. Anything that is `object` / `bool` / text will either have to be converted later or left out for now.

```
In [4]: spaceship.dtypes
```

```
Out[4]: PassengerId      str
HomePlanet      str
CryoSleep       object
Cabin           str
Destination     str
Age             float64
VIP             object
RoomService     float64
FoodCourt       float64
ShoppingMall    float64
Spa             float64
VRDeck          float64
Name            str
Transported     bool
dtype: object
```

```
In [5]: # a fuller view: dtypes + non-null counts in one go
spaceship.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8693 entries, 0 to 8692
Data columns (total 14 columns):
#   Column             Non-Null Count  Dtype
---  -
0   PassengerId         8693 non-null  str
1   HomePlanet          8492 non-null  str
2   CryoSleep           8476 non-null  object
3   Cabin               8494 non-null  str
4   Destination         8511 non-null  str
5   Age                 8514 non-null  float64
6   VIP                 8490 non-null  object
7   RoomService         8512 non-null  float64
8   FoodCourt           8510 non-null  float64
9   ShoppingMall        8485 non-null  float64
10  Spa                 8510 non-null  float64
11  VRDeck              8505 non-null  float64
12  Name                8493 non-null  str
13  Transported         8693 non-null  bool
dtypes: bool(1), float64(6), object(2), str(5)
memory usage: 1.2+ MB
```

Check for missing values

`.isna()` turns every cell into True/False, and `.sum()` counts the True s per column.

```
In [6]: spaceship.isna().sum()
```

```
Out[6]: PassengerId      0
        HomePlanet      201
        CryoSleep        217
        Cabin            199
        Destination      182
        Age              179
        VIP              203
        RoomService      181
        FoodCourt        183
        ShoppingMall      208
        Spa              183
        VRDeck           188
        Name             200
        Transported       0
        dtype: int64
```

There are multiple strategies to handle missing data

- Removing all rows or all columns containing missing data.
- Filling all missing values with a value (mean in continuous or mode in categorical for example).
- Filling all missing values with an algorithm.

For this exercise, because we have such low amount of null values, we will drop rows containing any missing value.

```
In [7]: spaceship = spaceship.dropna()
        spaceship.shape
```

```
Out[7]: (6606, 14)
```

KNN

K Nearest Neighbors is a distance based algorithm, and requires all **input data to be numerical**.

Let's only select numerical columns as our features.

`select_dtypes(include=np.number)` keeps only the numeric columns. Text columns (`Name`, `Cabin`, `HomePlanet` ...) are dropped for now — we will learn how to encode them into numbers later in the week.

```
In [8]: features = spaceship.select_dtypes(include=np.number)
        features.head()
```

```
Out [8]:
```

	Age	RoomService	FoodCourt	ShoppingMall	Spa	VRDeck
0	39.0	0.0	0.0	0.0	0.0	0.0
1	24.0	109.0	9.0	25.0	549.0	44.0
2	58.0	43.0	3576.0	0.0	6715.0	49.0
3	33.0	0.0	1283.0	371.0	3329.0	193.0
4	16.0	303.0	70.0	151.0	565.0	2.0

And also lets define our target.

The target is **Transported** — the column we want to predict. It is **not** part of the features.

```
In [9]: target = spaceship["Transported"]
target.head()
```

```
Out[9]: 0    False
1     True
2    False
3    False
4     True
Name: Transported, dtype: bool
```

Train Test Split

Now that we have split the data into **features** and **target** variables and imported the **train_test_split** function, split X and y into X_train, X_test, y_train, and y_test. 80% of the data should be in the training set and 20% in the test set.

test_size=0.2 → 20% test / 80% train. **random_state=17** → a seed, so the split (and therefore the score) is reproducible every run.

```
In [10]: X_train, X_test, y_train, y_test = train_test_split(
        features, target, test_size=0.2, random_state=17
    )

X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
Out[10]: ((5284, 6), (1322, 6), (5284,), (1322,))
```

Model Selection

In this exercise we will be using **KNN** as our predictive model.

You need to choose between **Classifier** or **Regressor**. Take into consideration target variable to decide.

Answer: Classifier. `Transported` is `True / False` — a **categorical** target with a finite number of outcomes. That makes this a **classification** problem, so we use `KNeighborsClassifier`. (A regressor would be for a continuous target, like a price or an age.)

Initialize a KNN instance without setting any hyperparameter.

```
In [11]: from sklearn.neighbors import KNeighborsClassifier

# no hyperparameters -> scikit-learn uses its default, n_neighbors=5
knn = KNeighborsClassifier()
knn
```

```
Out[11]: ▼ KNeighborsClassifier ⓘ ?
          ► Parameters
```

Fit the model to your data.

`fit` = train. Note that we only pass the **training** data here — the test set stays untouched, otherwise we would have **data leakage**.

```
In [12]: knn.fit(X_train, y_train)
```

```
Out[12]: ▼ KNeighborsClassifier ⓘ ?
          ► Parameters
          ► Fitted attributes
```

Evaluate your model.

`.score()` on a classifier predicts on `X_test`, compares with `y_test` and returns the **accuracy**.

```
In [13]: knn.score(X_test, y_test)
```

```
Out[13]: 0.7851739788199698
```

Extra — looking beyond accuracy

Accuracy alone can hide a bad model, so let's also look at the predictions, the confusion matrix and the precision / recall per class.

```
In [14]: predictions = knn.predict(X_test)
         predictions[:15]
```

```
Out[14]: array([ True,  True,  True,  True, False, False, False,  True,  True,
        True, False, False, False,  True,  True])
```

```
In [15]: from sklearn.metrics import confusion_matrix, classification_report
print(confusion_matrix(y_test, predictions))
```

```
[[474 156]
 [128 564]]
```

```
In [16]: print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
False	0.79	0.75	0.77	630
True	0.78	0.82	0.80	692
accuracy			0.79	1322
macro avg	0.79	0.78	0.78	1322
weighted avg	0.79	0.79	0.78	1322

Extra — is k = 5 the best choice?

`n_neighbors` is a hyperparameter, so we can test several values and compare.

```
In [17]: for k in [1, 3, 5, 10, 20, 50, 100]:
          model = KNeighborsClassifier(n_neighbors=k)
          model.fit(X_train, y_train)
          print(k, round(model.score(X_test, y_test), 4))
```

```
1 0.7254
3 0.7587
5 0.7852
10 0.7927
20 0.8011
50 0.8101
100 0.8041
```

Congratulations, you have just developed your first Machine Learning model!

Recap of what we did

Step	Code	Why
1 · Explore	<code>.shape</code> , <code>.dtypes</code> , <code>.isna().sum()</code>	know the data before modelling
2 · Clean	<code>.dropna()</code>	KNN cannot work with missing values
3 · Features / target	<code>select_dtypes(np.number)</code> , <code>["Transported"]</code>	X = predictors, y = what we predict

Step	Code	Why
4 · Split	<code>train_test_split(..., test_size=0.2)</code>	keep an untouched test set — no data leakage
5 · Instantiate	<code>KNeighborsClassifier()</code>	classification, because the target is categorical
6 · Fit	<code>.fit(X_train, y_train)</code>	train only on the training set
7 · Score	<code>.score(X_test, y_test)</code>	accuracy on unseen data

Ideas to improve it later: scale the numeric columns (KNN is distance based), encode the categorical columns instead of dropping them, and tune `n_neighbors` .